

GRUPO 17

API PERFORMANCE MONITOR

An Observability Platform for Web Services

Licenciatura Engenharia Informática e de Computadores

Projeto e Seminário

Francisco Dias, No. 51562

Martim Ferreira, No. 51755

Orientador * Pedro Pereira

O QUE É O PROJETO?



MOTIVAÇÃO / PROBLEMA

As APIs são componentes críticos nos sistemas modernos, sendo responsáveis pela comunicação entre serviços e aplicações.

No entanto, problemas como latência elevada, falhas ou indisponibilidade são frequentemente difíceis de detetar em tempo real.



O PROJETO

O API Performance Monitor é uma plataforma de observabilidade que permite monitorizar endpoints HTTP, recolher métricas como latência, uptime e erros, e analisar o comportamento das APIs ao longo do tempo.



SOLUÇÃO

A solução proposta consiste num sistema composto por um backend central, responsáveis pela gestão e análise de dados, e por workers de monitorização que executam pedidos periódicos às APIs, suportando ainda a monitorização de serviços públicos e privados.

REQUISITOS & HISTÓRIAS



REQUISITOS FUNCIONAIS

O sistema permite o registo e autenticação de utilizadores, a criação e gestão de endpoints monitorizados e a recolha de métricas associadas a cada endpoint, incluindo latência, código de estado e disponibilidade, garantindo isolamento por utilizador.



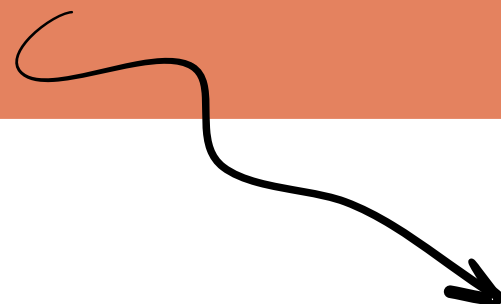
HISTÓRIAS DE UTILIZAÇÃO

Um utilizador pode registar-se, autenticar-se na plataforma e adicionar endpoints que pretende monitorizar. Após configuração, o sistema executa verificações periódicas e disponibiliza métricas sobre o desempenho desses serviços.

ARQUITETURA TECNOLOGIAS

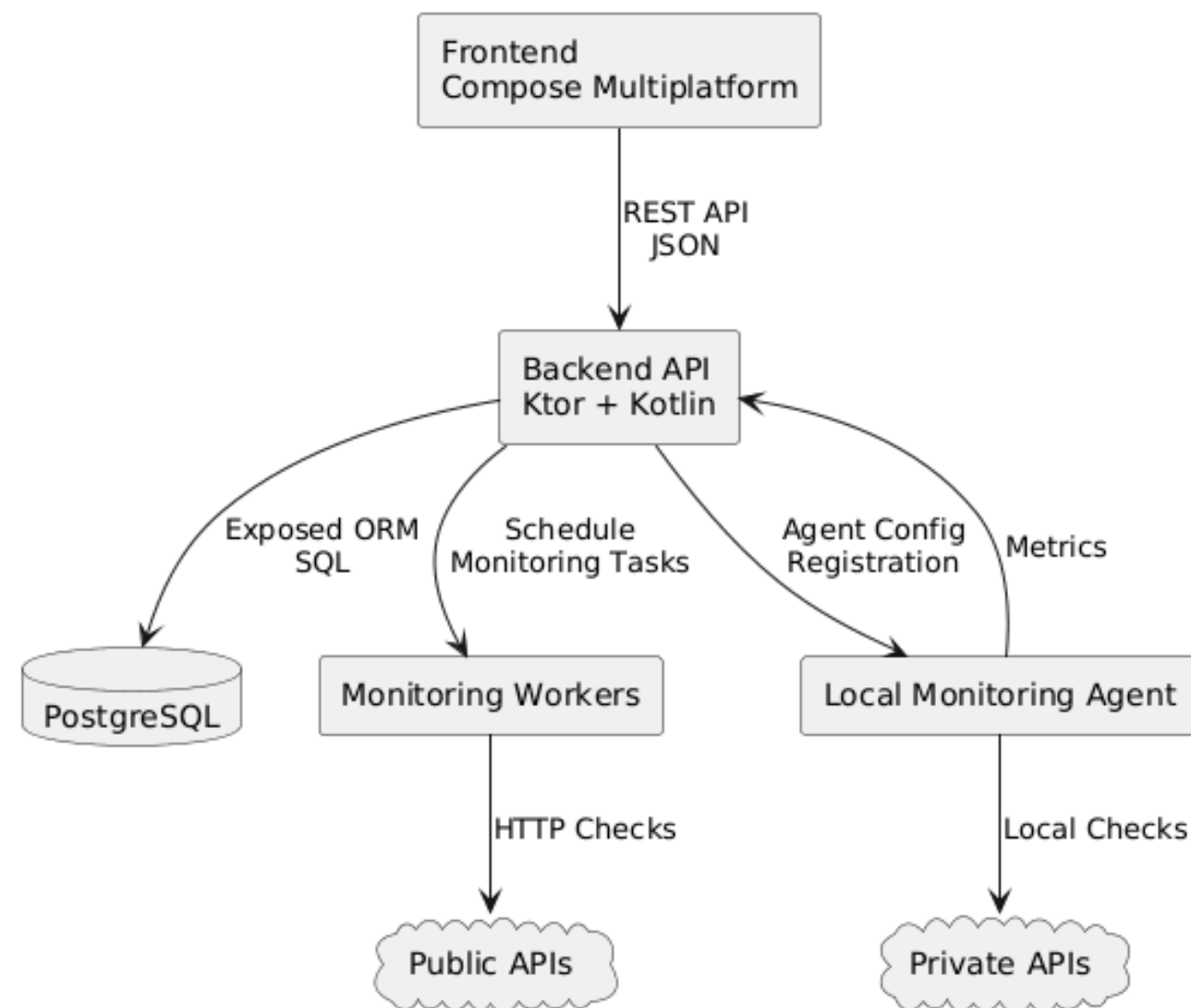
○ sistema está organizado em vários componentes:

- Um **cliente** multiplataforma desenvolvido em Compose Multiplatform para interação com o utilizador;
- Um **backend** desenvolvido em Kotlin + Ktor responsável pela lógica de negócio, autenticação e gestão de métricas;
- Um **conjunto de workers** assíncronos responsáveis pela monitorização periódica e recolha de métricas dos endpoints.



Permitindo recolher métricas de disponibilidade, latência e estado das APIs monitorizadas.

ARQUITETURA TECNOLOGIAS



O **sistema foi desenvolvido** utilizando Kotlin, com Ktor no backend e Compose Multiplatform no cliente. A comunicação é feita através de uma API REST e atualmente o sistema suporta armazenamento em memória e integração posterior com PostgreSQL.

- ➔ **Frontend:** Compose Multiplatform
- ➔ **Backend:** Kotlin + Ktor
- ➔ **Base de Dados:** H2
- ➔ **Comunicação:** HTTP REST + JSON
- ➔ **Autenticação:** JWT
- ➔ **Monitorização:** Workers + agentes locais
- ➔ **Notificações:** Discord Webhooks / Email

O sistema foi desenvolvido utilizando Kotlin Multiplatform (KMP), permitindo partilha de código entre cliente, backend e componentes auxiliares.

DIAGRAMA

DE

NAVEGAÇÃO

O sistema foi estruturado com uma navegação simples e orientada ao fluxo principal de utilização.

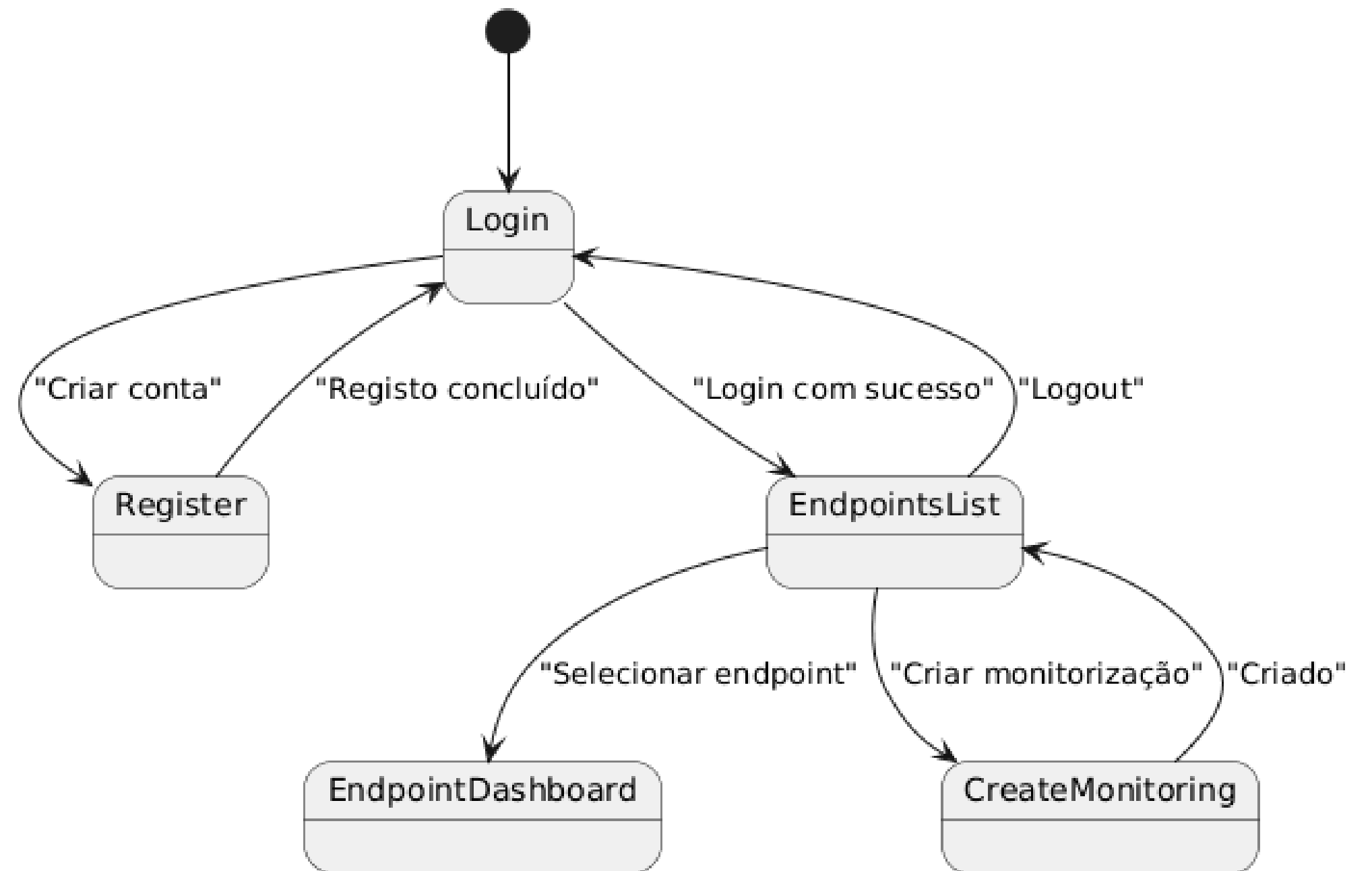


O utilizador inicia na autenticação, podendo efetuar login ou criar uma nova conta. Após autenticação bem-sucedida, é apresentada a lista de endpoints monitorizados, onde é possível consultar monitorizações existentes ou criar novas monitorizações.



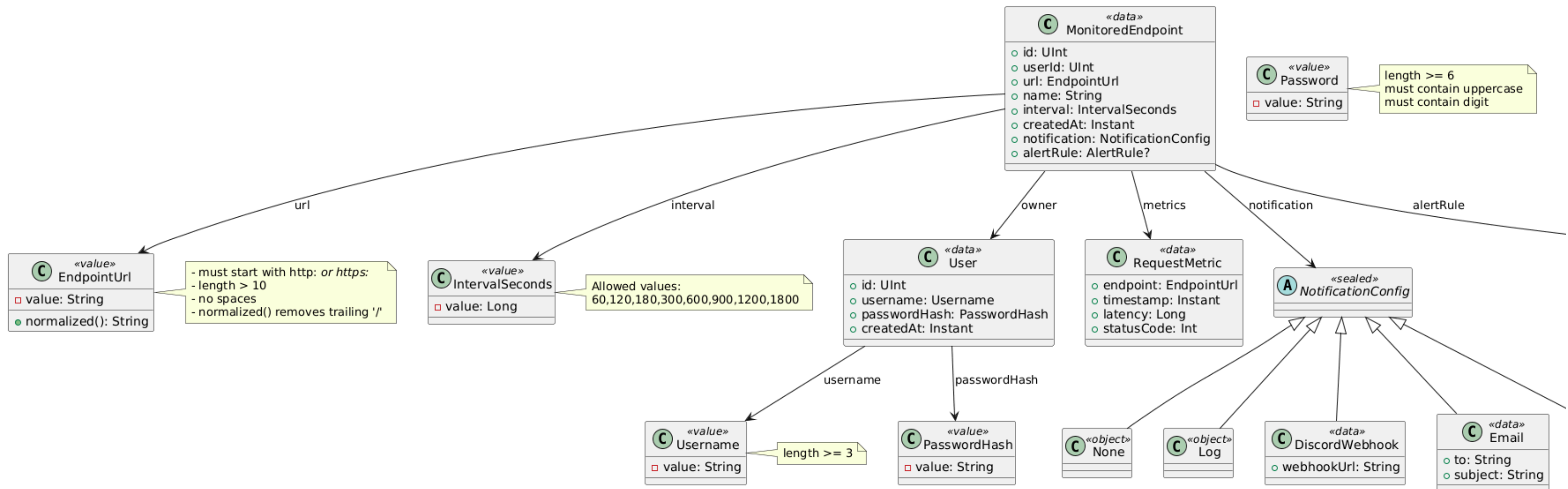
Cada endpoint possui um dashboard dedicado, permitindo visualizar métricas de desempenho, disponibilidade e estado das APIs monitorizadas.

Navigation Diagram



MODELO DE DADOS

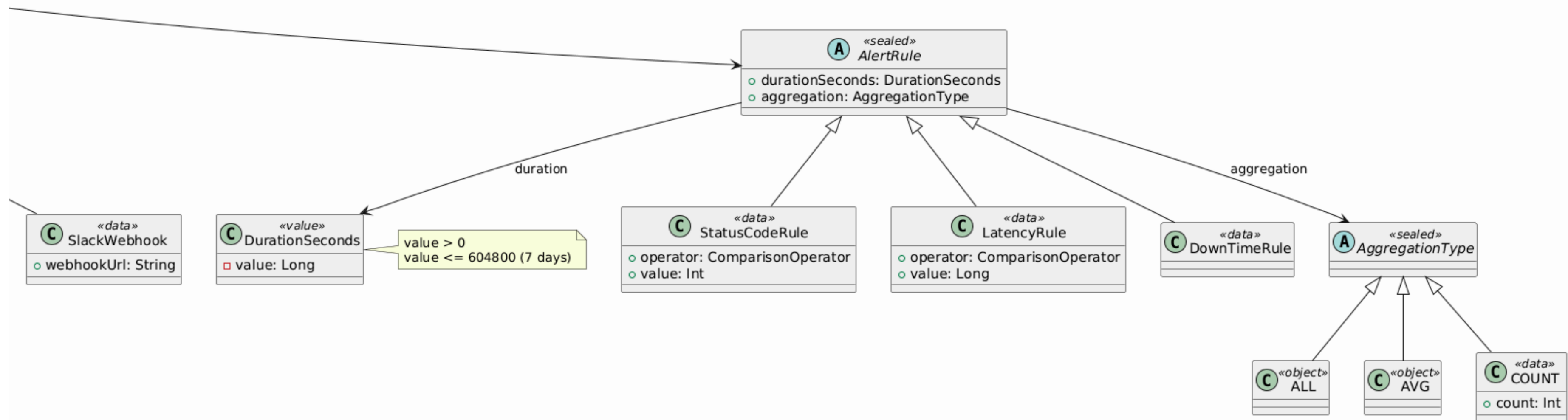
O modelo de dados inclui três entidades principais: utilizadores, endpoints monitorizados e métricas de requests. Estas entidades estão relacionadas de forma a permitir associar métricas a endpoints e endpoints a utilizadores.



MODELO DE DADOS



O modelo de dados inclui três entidades principais: utilizadores, endpoints monitorizados e métricas de requests. Estas entidades estão relacionadas de forma a permitir associar métricas a endpoints e endpoints a utilizadores.



O sistema foi desenvolvido com uma abordagem centrada no domínio (Domain-Driven Design), garantindo separação clara entre camadas e consistência nos dados através de value objects.

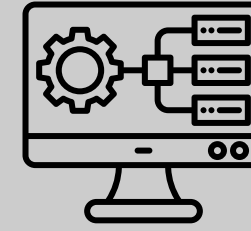
A autenticação é baseada em JWT, assegurando comunicação segura entre cliente e backend.

A monitorização é realizada de forma assíncrona por workers que executam pedidos periódicos aos endpoints, permitindo recolher métricas como latência e disponibilidade sem bloquear o sistema principal.

A arquitetura foi desenhada para ser modular e extensível, facilitando a evolução futura do sistema.

06

ASPETOS TÉCNICOS RELEVANTES



Separação em camadas e uso de Domain-Driven Design



Autenticação baseada em JWT



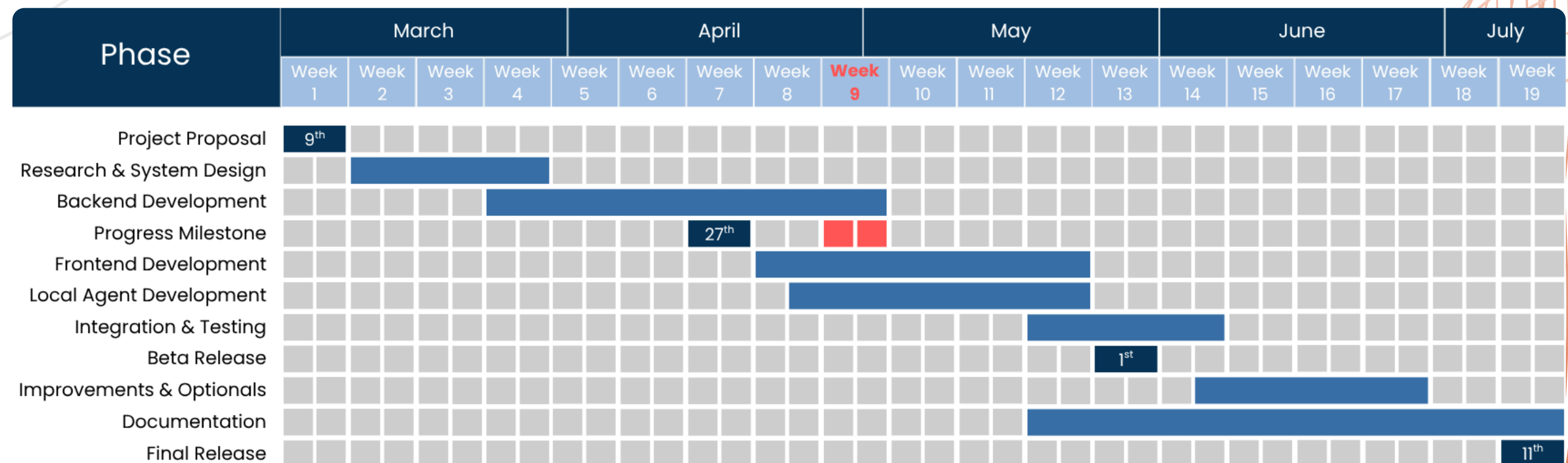
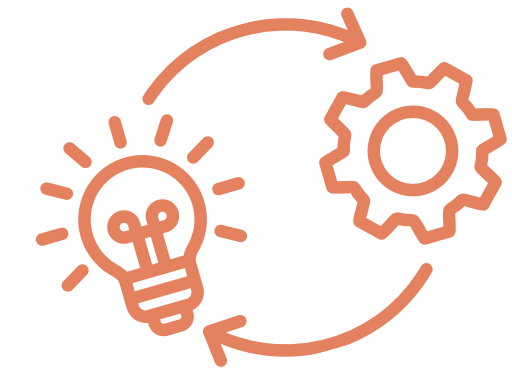
Monitorização assíncrona com workers

O QUE FALTA FAZER E COMO SERÁ FEITO

1. Finalizar a navegação e a interface do cliente
2. Garantir uma experiência de utilização consistente e integrada com o backend
3. Consolidar a consistência da API e dos contratos cliente-servidor
4. Expandir o sistema de monitorização
5. Melhorar a recolha e visualização de dados de monitorização
6. Concluir a integração do agente local
7. Realizar testes end-to-end ao sistema completo
8. Efetuar refinamentos finais de estabilidade e qualidade
9. Preparar a solução para a entrega final

O QUE FALTA FAZER E COMO SERÁ FEITO

1. Finalização da interface e navegação do cliente
2. Consolidação da API e contratos cliente-servidor
3. Expansão do sistema de monitorização
4. Integração completa do agente local
5. Melhoria da visualização de dados de monitorização
6. Testes end-to-end
7. Refinamentos finais e validação da solução



THANK *You!*

Francisco Dias, No. 51562
Martim Ferreira, No. 51755

Orientador * Pedro Pereira